

Addressing underfitting and overfitting through validation curve

Procedure:

1. Load the breast cancer dataset.
2. Check the first few records of the dataset.
3. Separate the input variables and the diagnosis.
4. Convert the diagnosis into numerical values.
5. Divide the data into training data (80%) and testing data (20%).
6. Standardize the data and create a Logistic Regression model.
7. Calculate training and validation scores for different training sample sizes.
8. Plot the learning curve showing training and validation accuracy.
9. Choose different values of the Logistic Regression parameter C.
10. Calculate training and validation scores for each value of C.
11. Plot the validation curve showing training and validation accuracy.
12. Compare the curves to understand overfitting and underfitting.

Code:

```
import pandas as pd

df = pd.read_csv("C:\\Users\\Lenovo\\Downloads\\wdbc.csv", header=None)

df.head()

from sklearn.preprocessing import LabelEncoder

X = df.loc[:, 2:].values
y = df.loc[:, 1].values

le = LabelEncoder()
y = le.fit_transform(y)
le.transform(['M', 'B'])

from sklearn.model_selection import train_test_split
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20, random_state=1)
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.model_selection import learning_curve
```

```
from sklearn.pipeline import Pipeline
```

```
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.linear_model import LogisticRegression
```

```
pipe_lr = Pipeline([('scl', StandardScaler()),
```

```
                    ('clf', LogisticRegression(penalty='l2', random_state=0))])
```

```
train_sizes, train_scores, test_scores = learning_curve(estimator=pipe_lr,
```

```
                X=X_train, y=y_train,
```

```
                train_sizes=np.linspace(0.1, 1.0, 10),
```

```
                cv=10,
```

```
                n_jobs=1)
```

```
train_mean = np.mean(train_scores, axis=1)
```

```
train_std = np.std(train_scores, axis=1)
```

```
test_mean = np.mean(test_scores, axis=1)
```

```
test_std = np.std(test_scores, axis=1)
```

```
# Train Score
```

```
plt.plot(train_sizes, train_mean, color='blue', marker='o', markersize=5, label='training  
accuracy')
```

```
plt.fill_between(train_sizes,
```

```
                train_mean + train_std,
```

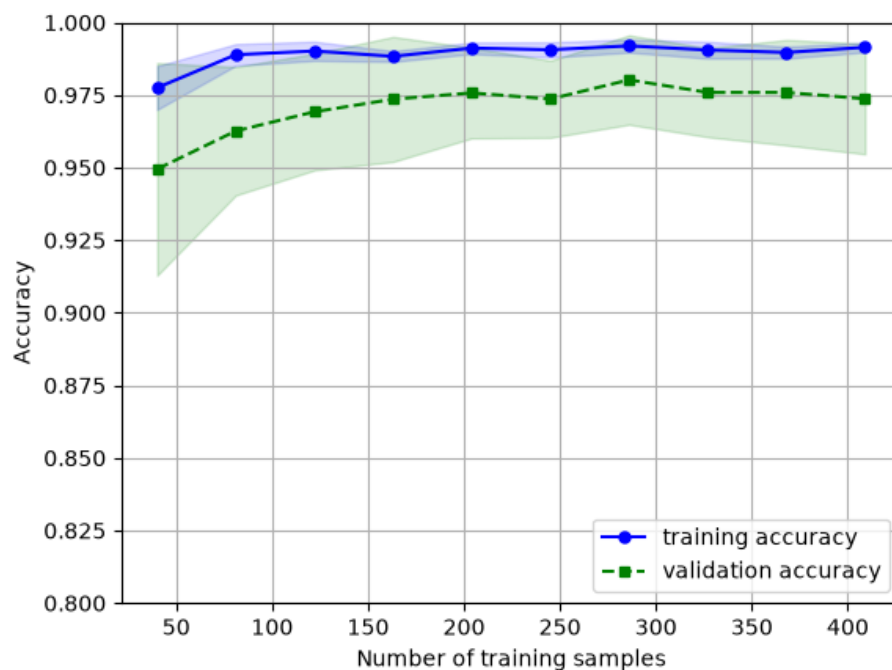
```
                train_mean - train_std,
```

```
                alpha=0.15, color='blue')
```

```
# Test Score
```

```
plt.fill_between(train_sizes,
                 test_mean + test_std,
                 test_mean - test_std,
                 alpha=0.15, color='green')
```

```
plt.grid()
plt.xlabel('Number of training samples')
plt.ylabel('Accuracy')
plt.legend(loc='lower right')
plt.ylim([0.8, 1.0])
plt.show()
```

[illegible]

```
param_name='clf__C',  
param_range=param_range,  
cv=10)
```

```
train_mean = np.mean(train_scores, axis=1)  
train_std = np.std(train_scores, axis=1)  
test_mean = np.mean(test_scores, axis=1)  
test_std = np.std(test_scores, axis=1)
```

```
plt.plot(param_range, train_mean, color='blue', marker='o', markersize=5, label='training  
accuracy')
```

```
plt.fill_between(param_range,  
                 train_mean + train_std,  
                 train_mean - train_std,  
                 alpha=0.15, color='blue')
```

```
plt.plot(param_range, test_mean, color='green', linestyle='--', marker='s', markersize=5,  
label='validation accuracy')
```

```
plt.fill_between(param_range,  
                 test_mean + test_std,  
                 test_mean - test_std,  
                 alpha=0.15, color='green')
```

```
plt.grid()  
plt.xscale('log')  
plt.legend(loc='lower right')  
plt.xlabel('Parameter C')  
plt.ylabel('Accuracy')  
plt.ylim([0.8, 1.0])  
plt.show()
```

